

Week 3 – Purple Teaming

1.0 Nmap Attacks

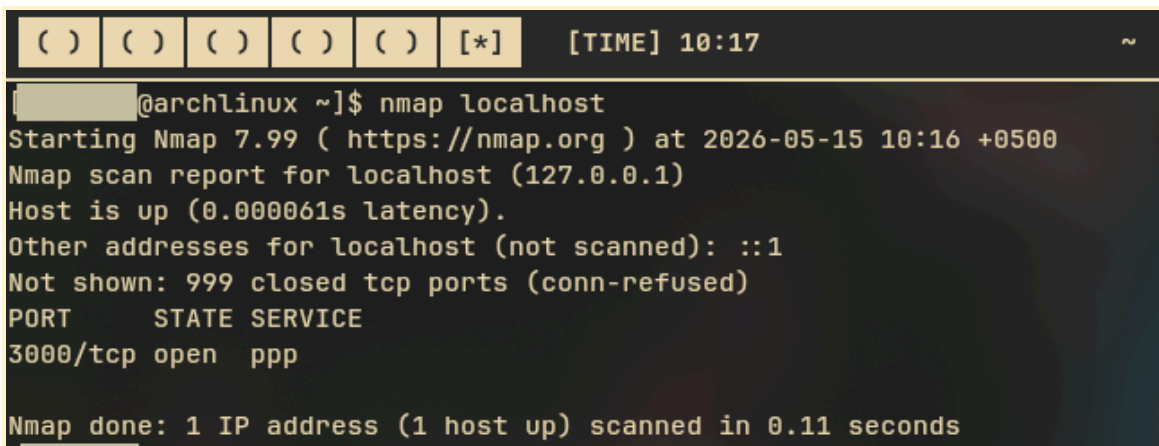
1.1 What Nmap does

Nmap works by sending specially crafted packets to a target system and analyzing the responses it receives. Based on those responses, it can determine which ports are open, which services are running, and sometimes even the operating system and software versions being used on the target machine. A few examples of attacks will be demonstrated below.

1.1.1 Basic Port Scan

Figure 1.1 Shows a basic port scan being performed by me using:

nmap localhost

A terminal window with a dark background and light-colored text. The window has a title bar with several icons and the text "[TIME] 10:17". The terminal content shows the command "nmap localhost" being executed. The output includes the Nmap version (7.99), the target IP (127.0.0.1), and a list of open ports (3000/tcp) and services (ppp).

```
( ) ( ) ( ) ( ) ( ) [*] [TIME] 10:17 ~
[redacted]@archlinux ~]$ nmap localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 10:16 +0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000061s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
3000/tcp  open  ppp
Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds
```

(Figure 1.1)

This scans common ports on your local machine and shows which services are open.

1.1.2 Specified Port Scan

Figure 1.2 Shows me picking port 3000 to scan, which is OWASP Juice Shop being hosted on the local host:

nmap -p 3000 localhost

```
[ ]@archlinux ~]$ nmap -p 3000 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 10:16 +0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000028s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE
3000/tcp  open  ppp

Nmap done: 1 IP address (1 host up) scanned in 0.03 seconds
```

(Figure 1.2)

This checks whether the web application port is accessible.

1.1.3 Vulnerability Port Scan

Figure 1.3 Shows a scan for common web vulnerabilities using Nmap's scripting engine:

nmap --script vuln localhost

```
[ ]@archlinux ~]$ nmap --script vuln localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 10:17 +0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000061s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
3000/tcp  open  ppp
```

(Figure 1.3)

This runs built-in vulnerability detection scripts against detected services.

1.1.3 Operating System (OS) Port Scan

Figure 1.3 Shows a scan for common web vulnerabilities using Nmap's scripting engine:

nmap -O localhost

```

[archlinux ~]$ sudo nmap -O localhost
[sudo] password for [redacted]:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-15 10:21 +0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000055s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
3000/tcp  open  ppp
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=5/15%OT=3000%CT=1%CU=32912%PV=Y%DS=0%DC=L%G=Y%TM=6A06A
OS:D52%P=x86_64-pc-linux-gnu)SEQ(SP=101%GCD=1%ISR=10C%TI=Z%CI=Z%II=I%TS=21)
OS:SEQ(SP=104%GCD=2%ISR=109%TI=Z%CI=Z%II=I%TS=21)SEQ(SP=104%GCD=2%ISR=10A%T
OS:I=Z%CI=Z%TS=21)SEQ(SP=107%GCD=1%ISR=10A%TI=Z%CI=Z%II=I%TS=20)SEQ(SP=FF%G
OS:CD=1%ISR=10B%TI=Z%CI=Z%TS=21)OPS(O1=MFFD7ST11NWA%O2=MFFD7ST11NWA%O3=MFFD
OS:7NNT11NWA%O4=MFFD7ST11NWA%O5=MFFD7ST11NWA%O6=MFFD7ST11)WIN(W1=FFCB%W2=FF
OS:CB%W3=FFCB%W4=FFCB%W5=FFCB%W6=FFCB)ECN(R=Y%DF=Y%T=40%W=FFD7%O=MFFD7NNSNW
OS:A%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF
OS:=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=
OS:%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=
OS:0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=6%RID=6%RI
OS:PCK=6%RUCK=6%RUD=6)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 0 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.45 seconds

```

(Figure 1.4)

This attempts OS fingerprinting by analyzing network responses.

2.0 Winston for Logging

2.1 Winston Usage

Winston allows applications to generate logs and store them in different locations, such as the terminal, text files, or external logging systems. In this project, Winston was configured to display logs in the console and save them into a security.log file. This made it possible to monitor login attempts and other security-related actions occurring within the application.

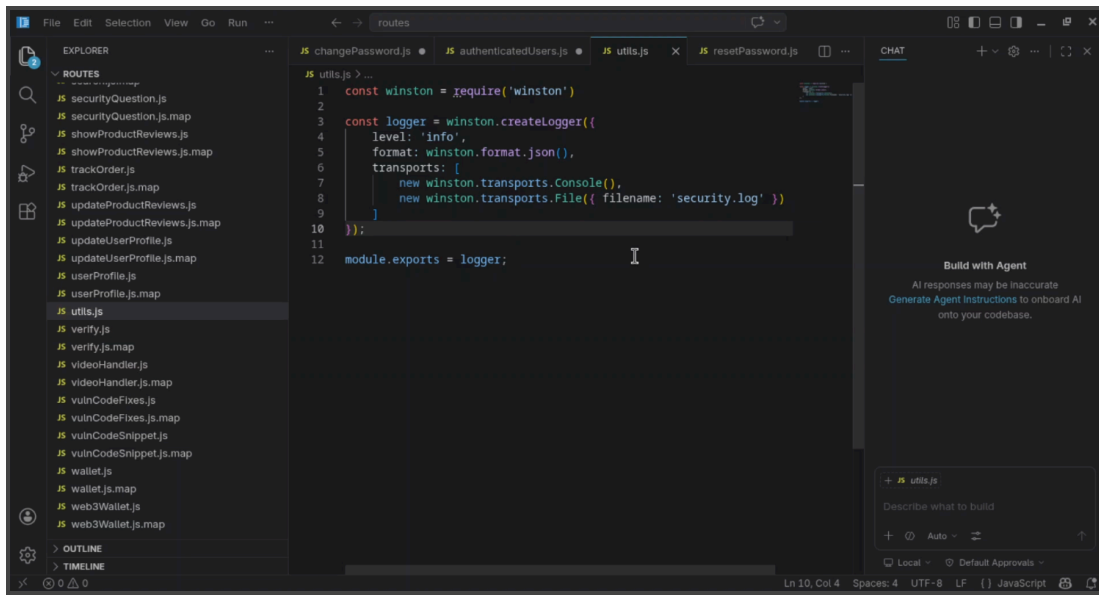
2.2 Installing Winston

To install winston, we will have to type the following command within the terminal, in the file where the local host is being hosted:

npm install winston

2.3 Implementing Winston

In order to implement a logging system, it is better practice to create a new file for this specific task. Hence, I created a new file named “utils.js”, that was created within the /build folder. I then wrote the following code in the file:

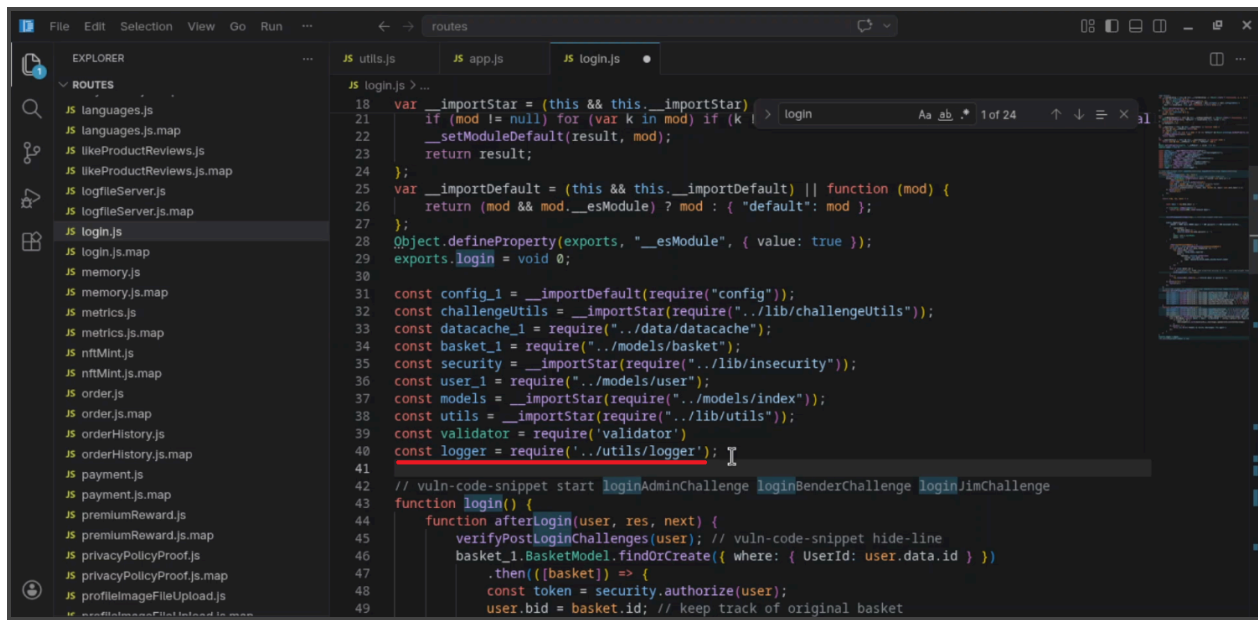


(Figure 1.1)

```
const logger = winston.createLogger ({
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'security.log' })
  ]
})
```

I changed my file location to login.js within the /build files and entered the following code:

```
const logger = require('../utils/logger')
```



(Figure 1.2)

The code underline in red is used to import a logger object from another file so it can be used inside the current file.

3.0 Checklist

I also created the following checklist to see how each of the following is implemented:

- ☐ Validated Inputs
- ☐ Salted and Hashed Inputs
- ☐ Protection from XSS
- ☐ HTTPS for Data Transmission

It can be observed that most of these were done in Week 2, hence - I shall document how they all have been implemented and further discuss what their impact is.

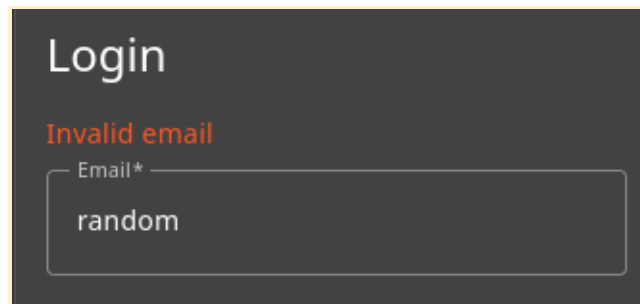
3.1 Validated Inputs

In order to validate inputs, we can write the following code to add a validator:

const validator = require('validator'); // Used to Import Validator into the File

```
If (!validator.isEmail(email)) {  
    return res.status(400).send('Invalid email')  
}
```

Figure 3.1 Shows how using values such as “random” (instead of the correct format of writing an email e.g “random@random.com”) prompts “Invalid email” as expected.



(Figure 3.1)

Hence, we can cross out Validated Inputs from the Checklist:

- ☒ ~~Validated Inputs~~
- ☐ Salted and Hashed Inputs
- ☐ Protection from XSS
- ☐ HTTPS for Data Transmission

3.2 Salted and Hashed Inputs

The `console.log('Password Has Been Hashed: ', hashedPassword);` line is what acts as an indicator that the password has been hashed and hence bcrypt is working. Which I tested this and it worked, since I received its output within my terminal (shown by the red underline):

```
info: Required file index.html is present (OK)
info: Required file styles.css is present (OK)
info: Required file main.js is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file matching /*hacking-instructor-.*\.js$/ is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
[blazryn@archlinux juice-shop]$ npm start

> juice-shop@19.2.1 start
> node build/app

info: Detected Node.js version v24.15.0 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file index.html is present (OK)
info: Required file styles.css is present (OK)
info: Required file main.js is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file matching /*hacking-instructor-.*\.js$/ is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Server listening on port 3000
info: Restored 1-star directoryListingChallenge (Confidential Document)
info: Restored 1-star localXssChallenge (DOM XSS)
info: Restored 1-star errorHandlingChallenge (Error Handling)
info: Restored 2-star loginAdminChallenge (Login Admin)
info: Restored 1-star scoreBoardChallenge (Score Board)
info: Restored 2-star exposedCredentialsChallenge (Exposed credentials)
Password Has Been Hashed: $2b$10$L8NQUREZuULNNTpdq2p8o0qHEzBSwnC8YHMc62PTnBmEy2ei243i
```

(Figure 3.2)

Hence, we can cross out Salted and Hashed Inputs from the Checklist:

- ☒ ~~Validated Inputs~~
- ☒ ~~Salted and Hashed Inputs~~
- ☐ Protection from XSS
- ☐ HTTPS for Data Transmission

3.3 Protection from XSS

The following code had a vulnerability – An SQL Injection attack could be performed using ‘*or 1=1 --*’. Hence, the following code was changed from:

```
models.sequelize.query('SELECT * FROM Users WHERE email = '${req.body.email} || '''  
AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL', {  
model: UserModel, plain: true })
```

To:

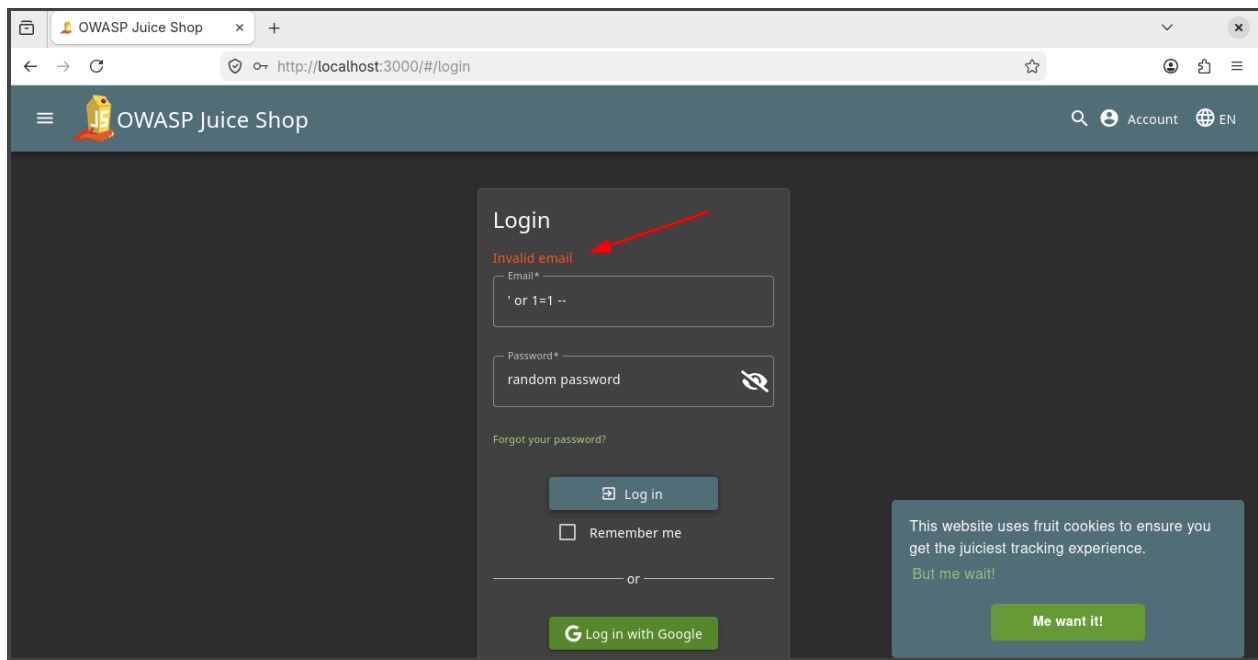
```
models.sequelize.query(  
  'SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS  
  NULL',  
  {  
    replacements: [  
      req.body.email || '',  
      security.hash(req.body.password || '')  
    ],  
    model: UserModel,  
    plain: true  
  }  
)
```

Side Note:

It is important to note that while validators can reduce the chances of an XSS attack (Since it requires the user to enter a specific format of data), It still is not completely safe from an XSS attack, so I added this as an extra measure so that there would be extra safety.

Continuing On:

Hence, when the same value ‘*or 1=1 --*’ was written as the email, It prompted the adversary that the following email was Invalid:



(Figure 1.3)

Hence, we can cross out Protection from XSS from the Checklist:

- ☒ ~~Validated Inputs~~
- ☒ ~~Salted and Hashed Inputs~~
- ☒ ~~Protection from XSS~~
- ☐ HTTPS for Data Transmission

3.3 Protection from XSS

When HTTPS is enabled, all communication between the web browser and the application server is encrypted before being sent across the network. This prevents attackers from performing common attacks such as packet sniffing, man-in-the-middle attacks, session hijacking, and credential theft. HTTPS also helps verify the authenticity of the server through digital certificates, ensuring that users are communicating with the legitimate application rather than a malicious impersonation.

In secure web application development, HTTPS is considered an essential security practice because many application vulnerabilities become significantly more dangerous when traffic is transmitted over unencrypted HTTP connections. Authentication systems, login forms, and API communication especially rely on HTTPS to protect user credentials and session information.

In this project, the application was primarily tested in a local development environment using *http://localhost:3000*, where HTTPS was not fully configured. However, the importance of HTTPS for protecting sensitive data transmission was reviewed and documented as part of the security improvement process. The project also explored related browser-side protections through the use of Helmet, which helps improve HTTP security headers and strengthen overall web application security practices.

Hence, we can cross out HTTPS for Data Transmission from the Checklist:

- ☒ ~~Validated Inputs~~
- ☒ ~~Salted and Hashed Inputs~~
- ☒ ~~Protection from XSS~~
- ☒ ~~HTTPS for Data Transmission~~

This concludes the assignments given across the 3 weeks completion time.